ASSIGNMENT – 2

Course: C++ and Object Oriented Programming

Register Number: 192524300

Student Name: Lakshmi Devi J.

GitHub Repository Link: https://github.com/laxmijaya0812-prog/DSA0119--C/tree/main

Assignment Title

Implementation and Evaluation of Advanced C++ Functions, References, Classes and Object-Oriented Programming Concepts

Q1 – Call by Value, by Reference and by Pointer

Marks: 6

1. Aim

To understand and implement call by value, call by reference and call by pointer in C++, and evaluate how each parameter-passing method affects the original variable.

2. Problem Description

A hostel office applies concessions to students' hostel fees. A function may receive the fee by value, reference or pointer.

The objective is to determine:

- Where each parameter is stored.
- Whether the original fee is modified.
- How references and pointers allow modification of caller data.
- When a const reference should be used.
- Whether modifying the fee directly or returning a new fee is a better design.

3. Call by Value

In call by value, a copy of the original value is passed to the function.

Any modification affects only the local copy.

Example

```
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}
```

If:

fee = 45000

and a 10% concession is applied, the local copy becomes:

45000 - 4500 = 40500

However, the original variable outside the function remains:

45000

Advantage

The original data cannot be accidentally changed.

Disadvantage

If the function is required to modify the original value, call by value is not suitable.

4. Call by Reference

In call by reference, the function parameter becomes another name for the original variable.

```
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
```

}

If fee is 45000, the original variable itself becomes:

45000 - 4500 = 40500

Therefore, call by reference is suitable when the function must update the caller's value.

5. Call by Pointer

In call by pointer, the address of the variable is passed.

```
void applyByPointer(long long* fee, int percent)
{
    *fee -= (*fee * percent) / 100;
}
```

The * operator accesses the value stored at the supplied address.

Therefore, modifying *fee modifies the original variable.

6. Comparison

Method	**Parameter**	**Original Variable Modified?**
Call by Value	Copy of value	No
Call by Reference	Reference	Yes
Call by Pointer	Address	Yes

7. When Should const Reference Be Used?

A const reference should be used when:

1. A large object should not be copied.
2. The function only needs to read the object.

3. The original object must not be modified.

Example:

```
void display(const string& name)
{
   cout << name;
}
```

The string is not copied and cannot be modified through the parameter.

8. Design Decision

When several concessions need to be applied, modifying the fee directly using a reference is suitable if the hostel system maintains one fee variable.

For example:

```
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
```

Reason

The function clearly represents an operation that updates the student's stored fee.

An alternative is to return the modified fee:

```
long long applyConcession(long long fee, int percent)
{
   return fee - fee * percent / 100;
}
```

This approach avoids side effects and may be easier to reason about.

9. Predict the Output

Given:

```
#include <iostream>
void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}
int& bigger(int& p, int& q)
{
    return p > q ? p : q;
}
void show(int x, int y = 5, int z = 10)
{
    std::cout << x + y + z << ' ';
}
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
```

```
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```



10. My Predicted Output

1 12 13

1 100

16 13 6

11. Explanation of Output

Initially:

a = 1

b = 2

c = 3

The call:

change(a, b, &c);

contains three different parameter-passing methods.

a

a is passed by value:

int a

Therefore, only a copy is changed.

Original:

a = 1

remains unchanged.

b

b is passed by reference:

int& b

Therefore:

2 + 10 = 12

The original b becomes 12.

c

The address of c is passed:

&c

Inside the function:

*c += 10;

Therefore:

3 + 10 = 13

So the first output is:

1 12 13

bigger(a, c) = 100

At this point:

a = 1

c = 13

The larger value is c.

The function returns a reference to c.

Therefore:

bigger(a, c) = 100;

changes:

c = 100

Hence:

1 100

Default Arguments

The function is:

void show(int x, int y = 5, int z = 10)

For:

show(1);

the default values are used:

x = 1

y = 5

z = 10

Therefore:

1 + 5 + 10 = 16

For:

show(1, 2);

only z uses the default value:

1 + 2 + 10 = 13

For:

show(1, 2, 3);

all values are supplied:

1 + 2 + 3 = 6

Therefore:

16 13 6

12. Complete the Code

The following function must modify the caller's fee.

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value   : "
```

```
        << hostelFee << '\n';
    applyConcession(hostelFee, 10);

    std::cout << "after call by reference: "
        << hostelFee << '\n';
    return 0;
}
```

```
Output

after call by value   : 45000
after call by reference: 40500
```

13. Expected Output

after call by value   : 45000

after call by reference: 40500

14. Alternative Concept

Instead of changing the original fee using a reference, the function can return the new fee.

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
```

Usage:

```
hostelFee = applyConcession(hostelFee, 10);
```

Comparison

Method	Original Modified	Return Value
Call by value	No	Not required
Call by reference	Yes	Not required
Return value	No directly	Yes

Returning a value is often preferable when the operation is conceptually a calculation rather than an in-place modification.

15. Complexity Analysis

The calculation performs a fixed number of operations.

Call by Value

- Time: O(1)
- Space: O(1)

Call by Reference

- Time: O(1)
- Space: O(1)

Call by Pointer

- Time: O(1)
- Space: O(1)

The difference is mainly in how the data is passed, not in asymptotic execution complexity.

16. Result

Thus, call by value, call by reference and call by pointer were implemented successfully. The experiment demonstrated that call by value modifies only a local copy, while call by reference and pointer can modify the original variable. Default arguments and returning references were also analysed.

Q2 – Return by Reference

Marks: 7

1. Aim

To implement a function that returns a reference to an array element and understand the advantages and risks of returning references.

2. Problem Description

A student's marks are stored in an array. The program must find the highest mark and return a reference to that element so that the original value can be modified.

3. Return by Reference

A function can return a reference using:

int&

Example:

```
int& highest(int marks[], int n)
{
   int index = 0;
   for (int i = 1; i < n; i++)
   {
      if (marks[i] > marks[index])
         index = i;
   }
   return marks[index];
}
```

The returned reference refers to the actual array element.

Therefore:

highest(marks, 4) += 2;

modifies the original array.

4. Complete the Code

```
#include <iostream>
using namespace std;
int& highest(int marks[], int n)
{
    int index = 0;
    for (int i = 1; i < n; i++)
    {
        if (marks[i] > marks[index])
            index = i;
    }
    return marks[index];
}
int main()
{
    int marks[] = {75, 91, 84, 88};
    highest(marks, 4) += 2;
    cout << "Highest mark: "
         << highest(marks, 4)
```

```
        << endl;
    return 0;
}
```

```
Output
Highest mark: 93
```

5. Expected Output

Highest mark: 93

6. Explanation

The array contains:

75 91 84 88

The highest value is 91.

The statement:

highest(marks, 4) += 2;

directly modifies that array element.

Therefore:

91 + 2 = 93

The original array becomes:

75 93 84 88

7. Danger of Returning a Reference to a Local Variable

Incorrect:

int& dangerous()

```
{
   int local = 50;
   return local;
}
```

The variable local is destroyed when the function finishes.

Therefore, the returned reference refers to an object that no longer exists.

This results in **undefined behavior**.

8. Safe Return by Reference

Returning a reference to an existing object or array element is safe if that object remains alive.

Example:

```
int& getFirst(int arr[])
{
   return arr[0];
}
```

The array exists outside the function, so its lifetime continues after the function returns.

9. Alternative Concept – Return Index

Instead of returning a reference, the function can return the index.

```
int highestIndex(int marks[], int n)
{
   int index = 0;
   for (int i = 1; i < n; i++)
   {
```

```
        if (marks[i] > marks[index])
            index = i;
    }
    return index;
}
```

Then:

```
int index = highestIndex(marks, 4);
marks[index] += 2;
```

This makes the modification explicit.

10. Complexity Analysis

Finding the largest element requires checking the elements once.

- Time Complexity: O(n)
- Space Complexity: O(1)

Both returning a reference and returning an index have the same asymptotic complexity.

11. Result

Thus, returning a reference was implemented successfully. The program modified the original array element through the returned reference. The danger of returning references to local variables was also identified, and returning an index was considered as an alternative.

Q3 – Default Arguments and Function Overloading

Marks: 7

1. Aim

To understand default arguments and function overloading in C++, identify ambiguity in overloaded functions, and redesign the courier charge system using a clear and efficient function interface.

2. Problem Description

A courier service calculates delivery charges based on:

- Parcel weight
- Express delivery
- Insurance

The system needs to support different types of input such as weight in kilograms, weight in grams, and a complete parcel object.

The program should use **default arguments** and **function overloading** without creating ambiguous function calls.

3. Default Arguments

A default argument is a value automatically assigned to a function parameter when the caller does not provide a value.

Example:

```
void display(int x, int y = 10)
{
   cout << x + y;
}
```

When we call:

display(5);

the compiler uses:

x = 5

y = 10

Therefore, the result is:

15

Default arguments are useful when a parameter usually has a standard value.

4. Function Overloading

Function overloading allows multiple functions to have the same name but different parameter lists.

Example:

```
int add(int a, int b)
{
   return a + b;
}

double add(double a, double b)
{
   return a + b;
}
```

The compiler determines which function to call based on the arguments supplied.

5. Problem with an Ambiguous Courier Design

Suppose several charge() functions are created with overlapping parameters and default arguments.

If more than one function can accept the same call, the compiler may not be able to determine which function should be selected.

For example, poorly designed overloads may make a call such as:

charge(2);

ambiguous.

Therefore, the overloads must be designed using clearly distinguishable parameter types.

6. Redesigned Courier System

```
#include <iostream>

using namespace std;

struct Parcel

{

   double weight;

   bool express;

   bool insured;

};

double charge(double weight,

              bool express = false,

              bool insured = false)

{

   double amount = weight * 50;

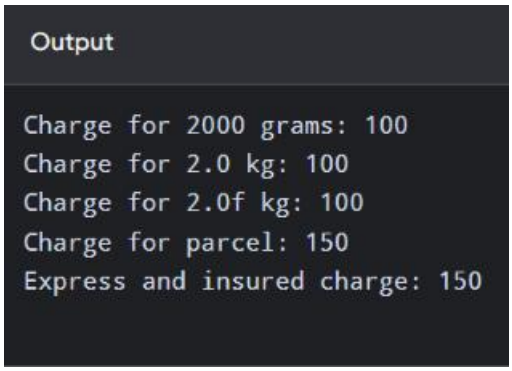
   if (express)

      amount += 30;
```

```
    if (insured)
        amount += 20;
    return amount;
}
double charge(int grams)
{
    double kg = grams / 1000.0;

    return charge(kg);
}
double charge(float kg)
{
    return charge(static_cast<double>(kg));
}
double charge(const Parcel& p)
{
    return charge(p.weight,
                  p.express,
                  p.insured);
}
int main()
{
    Parcel p{2.0, true, true};
    cout << "Charge for 2000 grams: "
```

```
        << charge(2000) << endl;
    cout << "Charge for 2.0 kg: "
        << charge(2.0) << endl;
    cout << "Charge for 2.0f kg: "
        << charge(2.0f) << endl;
    cout << "Charge for parcel: "
        << charge(p) << endl;
    cout << "Express and insured charge: "
        << charge(2.0, true, true)
        << endl;
    return 0;
}
```



7. Explanation of the Program

Parcel Structure

```
struct Parcel
{
    double weight;
    bool express;
    bool insured;
```

};

The structure stores information about one parcel.

- weight stores weight in kilograms.
- express indicates express delivery.
- insured indicates whether insurance is required.

Main Charge Function

double charge(double weight,

bool express = false,

bool insured = false)

This function calculates the basic courier charge.

The basic rate is:

Weight × 50

If express delivery is selected:

+ 30

If insurance is selected:

+ 20

Integer Overload

double charge(int grams)

This accepts weight in grams.

For example:

2000 grams = 2 kilograms

The conversion is:

double kg = grams / 1000.0;

Float Overload

double charge(float kg)

This accepts the weight as a float.

It converts the value to double and uses the main charge() function.

Parcel Overload

double charge(const Parcel& p)

This accepts a complete Parcel object.

It extracts the parcel information and calls the main charging function.

8. Predict the Overload Selection

Case 1

charge(2000);

2000 is an integer.

Therefore:

charge(int grams)

is selected.

Case 2

charge(2.0);

2.0 is a double.

Therefore:

charge(double weight,

bool express,

bool insured)

is selected.

The default values are:

express = false

insured = false

Case 3

charge(2.0f);

The f means the value is a float.

Therefore:

charge(float kg)

is selected.

Case 4

charge(p);

p is a Parcel object.

Therefore:

charge(const Parcel& p)

is selected.

Case 5

charge(2.0, true, true);

The compiler selects the double version because the first parameter is a double and the remaining two parameters are Boolean values.

9. Sample Calculation

For a parcel weighing 2 kg:

Basic charge = 2 × 50

= 100

For express delivery:

100 + 30 = 130

For insurance:

130 + 20 = 150

Therefore:

Express + insured charge = 150

10. Alternative Design

Instead of function overloading, different function names can be used.

Example:

chargeByGrams()

chargeByKilograms()

chargeParcel()

Advantage

The function purpose is immediately clear.

Disadvantage

More function names are required.

Function overloading provides a common interface such as:

charge()

while allowing different input types.

11. Complexity Analysis

The calculation uses a fixed number of arithmetic operations.

Time Complexity: O(1)

Space Complexity: O(1)

12. Result

Thus, default arguments and function overloading were successfully implemented. The courier charge system was redesigned to avoid ambiguous overloads, and different types of input were handled using suitable overloaded functions.

Q4 – Class with Array: Savings Account Mini-Statement

Marks: 7

1. Aim

To implement a C++ class containing a private array for storing recent bank transactions and understand encapsulation, fixed-size storage, insertion, deletion and complexity.

2. Problem Description

A savings account maintains a mini-statement containing the latest 10 transactions.

The class must:

- Store the account number.
- Store the current balance.
- Store the latest transactions.
- Add new transactions.
- Display the transaction history.
- Remove the oldest transaction when the array becomes full.

3. Class with an Array

An array can be declared as a data member inside a class.

double transactions[10];

This allows the object to maintain transaction information internally.

The array is declared as private so that external code cannot directly modify the transaction history.

4. Why is the Array Private?

The following declaration:

```
private:
    double transactions[10];
```

provides **encapsulation**.

Only member functions of the class can directly access the array.

For example, the following is not allowed:

```
account.transactions[0] = 50000;
```

because transactions is private.

Instead, the transaction must be added through:

```
account.addTransaction(50000);
```

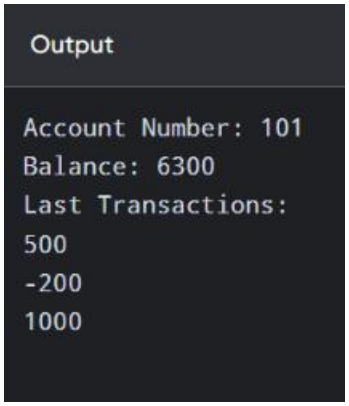
This allows the class to control how transactions are added.

5. Complete Program

```
#include <iostream>
using namespace std;
class SavingsAccount
{
private:
    int accountNumber;
    double balance;
    double transactions[10];
    int count;
public:
    SavingsAccount(int acc, double bal)
    {
```

```
    accountNumber = acc;
    balance = bal;
    count = 0;
}
void addTransaction(double amount)
{
    if (count < 10)
    {
        transactions[count] = amount;
        count++;
    }
    else
    {
        for (int i = 1; i < 10; i++)
        {
            transactions[i - 1] =
                transactions[i];
        }
        transactions[9] = amount;
    }
    balance += amount;
}
void display()
{
```

```
        cout << "Account Number: "
             << accountNumber << endl;
        cout << "Balance: "
             << balance << endl;
        cout << "Last Transactions:"
             << endl;
        for (int i = 0; i < count; i++)
        {
            cout << transactions[i]
                 << endl;
        }
    }
};
int main()
{
    SavingsAccount account(101, 5000);
    account.addTransaction(500);
    account.addTransaction(-200);
    account.addTransaction(1000);
    account.display();
    return 0;
}
```



6. Explanation of the Program

Private Data Members

int accountNumber;

double balance;

double transactions[10];

int count;

These variables store the account information.

Constructor

SavingsAccount(int acc, double bal)

The constructor initializes the account number, balance and transaction count.

Initially:

count = 0

because no transactions have been stored.

Adding a Transaction

void addTransaction(double amount)

This function adds a new transaction.

If the array still has space:

transactions[count] = amount;

count++;

The transaction is stored at the next available position.

7. What Happens When the 11th Transaction is Added?

The array contains only 10 positions.

Suppose it contains:

100

200

300

400

500

600

700

800

900

1000

When 1100 is added, there is no empty position.

Therefore, the program shifts the transactions:

200 → Position 1

300 → Position 2

400 → Position 3

...

1000 → Position 9

Then:

1100 → Position 10

The result becomes:

200

300

400

500

600

700

800

900

1000

1100

Thus, the oldest transaction 100 is removed.

8. Why is Shifting Required?

A normal array stores elements in consecutive memory locations.

When the oldest element must be removed, the remaining elements are shifted to maintain the order.

This requires multiple operations.

Therefore, when the array is full, insertion becomes O(n).

9. Change the System to 50 Transactions

The array can be changed to:

double transactions[50];

However, when the 51st transaction is added, the program would need to shift up to 49 elements.

This is inefficient for a system that continuously receives transactions.

10. Alternative Concept – Circular Buffer

A circular buffer can be used instead of shifting elements.

A circular buffer treats the array as circular.

When the end is reached, the next insertion returns to the beginning.

For example:

Position:

0 1 2 3 4 5 6 7 8 9

↑ ↑

Start End

When the array becomes full, the oldest transaction can be overwritten directly.

Advantage

Insertion can be performed in:

O(1)

instead of:

O(n)

11. Complexity Analysis

Normal insertion when space exists

O(1)

Insertion when the array is full

O(n)

because shifting is required.

Displaying transactions

O(n)

Space Complexity

O(n)

12. Result

Thus, a Savings Account class with a private transaction array was implemented successfully. Encapsulation was demonstrated by restricting direct access to the transaction array. The limitation of fixed-array shifting was identified, and a circular buffer was proposed as a more efficient alternative.

Q5 – Static Members and Arrays of Objects

Marks: 7

1. Aim

To understand static data members, static member functions and arrays of objects by implementing a bus ticket reservation system with a fixed capacity.

2. Problem Description

A bus has a maximum capacity of 3 passengers.

The system must:

- Create ticket objects.
- Allocate seats.
- Maintain the number of issued tickets.
- Prevent more than 3 seats from being issued.
- Display the waiting status of additional passengers.

The ticket count must be shared by all objects.

3. Static Data Member

A static data member belongs to the class rather than an individual object.

Example:

static int issued;

Only one copy exists for the entire class.

All objects share the same variable.

4. Complete Program

#include <iostream>

```
#include <string>
using namespace std;
class BusTicket
{
private:
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;
public:
    BusTicket(const string& n)
    {
        name = n;
        seat = 0;
        if (issued < CAPACITY)
        {
            ++issued;
            seat = issued;
        }
    }
    static int count()
    {
        return issued;
    }
```

```
    void print() const
    {
        if (seat != 0)
        {
            cout << name
                 << ": Seat "
                 << seat << endl;
        }
        else
        {
            cout << name
                 << ": Waiting" << endl;
        }
    }
};

int BusTicket::issued = 0;
int main()
{
    BusTicket b1("Asha");
    BusTicket b2("Bala");
    BusTicket b3("Chitra");
    BusTicket b4("Dev");
    b1.print();
```

```
    b2.print();
    b3.print();
    b4.print();
    cout << "Issued: "
         << BusTicket::count()
         << endl;
    return 0;
}
```

```
Output

Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5. Explanation

Initially:

issued = 0

First Object

BusTicket b1("Asha");

Since:

issued < 3

a seat is assigned.

issued = 1

seat = 1

Second Object

BusTicket b2("Bala");

Now:

issued = 2

seat = 2

Third Object

BusTicket b3("Chitra");

Now:

issued = 3

seat = 3

Fourth Object

BusTicket b4("Dev");

The capacity is already 3.

Therefore:

Dev → Waiting

No additional seat is issued.

6. Expected Output

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

7. Why is issued Static?

If issued were an ordinary member, every object would have its own copy.

For example:

Asha → issued = 1

Bala → issued = 1

Chitra → issued = 1

This would not correctly maintain the total number of issued tickets.

Using:

static int issued;

creates one shared counter:

Asha → issued = 1

Bala → issued = 2

Chitra → issued = 3

Dev → issued = 3

Therefore, static members are suitable for information shared by all objects.

8. Static Member Function

The function:

static int count()

can be called using the class name:

BusTicket::count();

It does not require a particular object.

9. Static Counter Across Function Calls

Static variables inside functions retain their values between function calls.

Example:

```
void createTicket()
{
   static int count = 0;
   count++;
   cout << count << endl;
}
```

Calling:

```
createTicket();
createTicket();
createTicket();
```

produces:

1

2

3

The variable is initialized only once.

10. Alternative Design

A separate TicketManager class can maintain:

- Total capacity
- Number of tickets issued
- Seat allocation
- Waiting list

This is useful for a larger reservation system because ticket data and reservation management can be separated.

11. Complexity Analysis

Ticket creation performs a fixed number of operations.

Time Complexity: O(1)

Space Complexity: O(n) for n ticket objects.

12. Result

Thus, static data members and static member functions were implemented successfully. The shared ticket counter correctly maintained the total number of issued tickets, and the capacity restriction prevented additional seats from being allocated.

Q6 – Friend Functions and Inline Functions

Marks: 8

1. Aim

To implement friend functions for accessing private members of a Time class, understand inline functions and compare a friend-function design with a member-function design.

2. Problem Description

A Time class stores hours and minutes as private data.

The system requires functions to:

1. Calculate the difference between two times.
2. Determine which time occurs earlier.

The functions are implemented as friend functions.

3. Friend Function

A friend function is a non-member function that is allowed to access the private members of a class.

It is declared inside the class using:

friend

Example:

```
class Test
{
private:
   int value;
public:
   friend void display(Test);
```

```
};
```

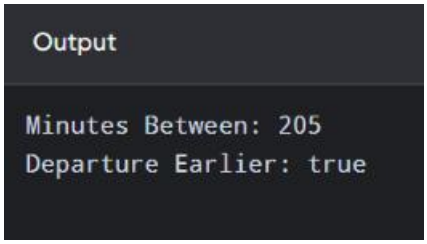
The friend function can access value.

4. Complete Program

```
#include <iostream>
using namespace std;
class Time
{
private:
   int h;
   int m;
public:
   Time(int hh, int mm)
   {
      h = hh;
      m = mm;
   }
   friend int minutesBetween(
      const Time& a,
      const Time& b);
   friend bool earlier(
      const Time& a,
      const Time& b);
};
```

```
int minutesBetween(
    const Time& a,
    const Time& b)
{
    int timeA = a.h * 60 + a.m;
    int timeB = b.h * 60 + b.m;
    if (timeA > timeB)
        return timeA - timeB;
    return timeB - timeA;
}
bool earlier(
    const Time& a,
    const Time& b)
{
    int timeA = a.h * 60 + a.m;
    int timeB = b.h * 60 + b.m;
    return timeA < timeB;
}
int main()
{
    Time departure(9, 45);
    Time arrival(13, 10);
    cout << "Minutes Between: " << minutesBetween(departure,  arrival)  << endl;
    cout << "Departure Earlier: " << boolalpha << earlier(departure,  arrival) << endl;
```

```
    return 0;
}
```



5. Explanation

Time Class

The class contains:

int h;

int m;

These are private members.

They cannot be accessed directly from outside the class.

minutesBetween()

The function converts both times into total minutes.

For example:

9:45

becomes:

9 × 60 + 45

= 585 minutes

Similarly:

13:10

becomes:

13 × 60 + 10

= 790 minutes

The difference is:

790 - 585 = 205 minutes

earlier()

The function compares the total minutes.

585 < 790

Therefore:

true

is returned.

6. Expected Output

Minutes Between: 205

Departure Earlier: true

7. Test Cases

Test Case 1

Time 1 = 09:00

Time 2 = 09:00

Result:

0 minutes

Both times are equal.

Test Case 2

Time 1 = 09:00

Time 2 = 11:30

Calculation:

9 × 60 + 0 = 540

11 × 60 + 30 = 690

690 - 540 = 150

Result:

150 minutes

Test Case 3

Time 1 = 11:30

Time 2 = 12:15

Calculation:

11 × 60 + 30 = 690

12 × 60 + 15 = 735

735 - 690 = 45

Result:

45 minutes

8. Inline Function

An inline function is a function declared using the inline keyword.

Example:

```
inline int square(int x)
{
   return x * x;
}
```

When the function is called, the compiler may replace the function call with the function body.

This can reduce function-call overhead for small, frequently used functions.

However, inline is only a request to the compiler. The compiler decides whether to perform the optimization.

9. Member Function Alternative

Instead of using friend functions, the operations can be implemented as member functions.

Example:

```
class Time
{
private:
   int h;
   int m;
public:
   Time(int hh, int mm)
   {
      h = hh;
      m = mm;
   }
   int totalMinutes() const
```

```
    {
        return h * 60 + m;
    }
    int minutesBetween(
        const Time& other) const
    {
        int a = totalMinutes();
        int b = other.totalMinutes();


        return (a > b) ? a - b : b - a;
    }
    bool earlier(
        const Time& other) const
    {
        return totalMinutes()
            < other.totalMinutes();
    }
};
```

Advantage

The operations are directly associated with the Time object.

This is generally a more object-oriented design when the operation naturally belongs to the class.

10. Friend Function vs Member Function

Feature	Friend Function	Member Function
Class member?	No	Yes
Can access private data?	Yes	Yes
Called using object?	Usually no	Yes
Encapsulation	Slightly weaker	Stronger
Suitable for symmetric operations	Yes	Sometimes

11. Complexity Analysis

Converting hours and minutes to total minutes requires a fixed number of operations.

Time Complexity: O(1)

Space Complexity: O(1)

12. Performance Measurement

The assignment requires measuring a large number of function calls under different compiler optimization levels.

Example:

```
#include <iostream>

#include <chrono>

using namespace std;

inline int add(int a, int b)

{

   return a + b;

}

int main()
```

```
{
    const long long N = 10000000;
    long long result = 0;
    auto start =
        chrono::high_resolution_clock::now();
    for (long long i = 0; i < N; i++)
    {
        result += add(i, 1);
    }

    auto end =  chrono::high_resolution_clock::now();
    auto duration =  chrono::duration_cast< chrono::microseconds>( end - start);
    cout << "Result: " << result << endl;
    cout << "Execution Time: "<< duration.count() << " microseconds"<< endl;
    return 0;
}
```

Observation

The -O2 version is generally expected to execute faster because the compiler applies optimization techniques.

The exact execution time depends on:

- Processor
- Compiler version
- Operating system
- Background applications

- System load

Therefore, actual measured values should be used in the final report.

16. Result

Thus, friend functions were successfully implemented to access private members of the Time class. The time difference and earlier-time operations were validated using multiple test cases. Inline functions and compiler optimization levels were also studied.

Q7 – Diagnose, Redesign, Validate and Defend

Marks: 8

1. Aim

To identify a limitation in an earlier C++ program, define a measurable improvement objective, redesign the program using a suitable data structure, implement the solution and validate its performance.

2. Selected Program

The **Savings Account transaction program from Q4** is selected.

3. Existing Design

The original program uses:

double transactions[10];

When the array becomes full, a new transaction requires shifting all existing transactions.

4. Problem / Limitation

The main limitation is inefficient insertion when the transaction array is full.

For example:

100

200

300

400

500

600

700

800

900

1000

When 1100 is added, the program must shift:

200 → first position

300 → second position

400 → third position

...

1000 → ninth position

Then:

1100 → tenth position

This requires multiple operations.

5. Measurable Objective

The redesigned system must:

1. Store only the latest 10 transactions.
2. Remove the oldest transaction when full.
3. Avoid shifting existing elements.
4. Insert a new transaction in O(1) time.
5. Use fixed memory.

6. Proposed Solution

A **circular buffer** is used.

The circular buffer stores transactions in a fixed-size array.

When the end of the array is reached, the next position wraps around to the beginning.

7. Important Variables

int start;

int count;

start

Stores the index of the oldest transaction.

count

Stores the number of currently stored transactions.

The next insertion position is calculated using:

(start + count) % SIZE

The modulo operator makes the array circular.

8. Redesigned Program

```
#include <iostream>

using namespace std;

class SavingsAccount

{

private:

    static const int SIZE = 10;

    double transactions[SIZE];

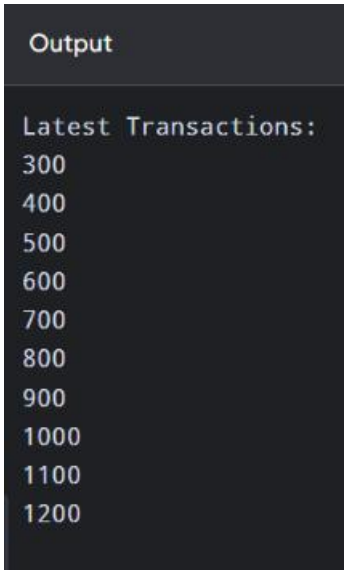
    int start;

    int count;

public:
```

```
SavingsAccount()
{
   start = 0;
   count = 0;
}
void addTransaction(double amount)
{
   int position =
      (start + count) % SIZE;
   if (count < SIZE)
   {
      transactions[position] =
         amount;
      count++;
   }
   else
   {
      transactions[start] =
         amount;
      start =
         (start + 1) % SIZE;
   }
}
void display()
```

```
    {
        cout << "Latest Transactions:" << endl;
        for (int i = 0; i < count; i++)
        {
            int position =(start + i) % SIZE;
            cout << transactions[position]
                 << endl;
        }
    }
};
int main()
{
    SavingsAccount account;
    for (int i = 1; i <= 12; i++)
    {
        account.addTransaction(i * 100);
    }
    account.display();
    return 0;
}
```



9. Working of the Circular Buffer

Twelve transactions are added:

100

200

300

400

500

600

700

800

900

1000

1100

1200

The capacity is only 10.

After inserting the first 10:

100

200

300

400

500

600

700

800

900

1000

When 1100 is added, the oldest value 100 is replaced.

Now:

200

300

400

500

600

700

800

900

1000

1100

When 1200 is added, 200 is replaced.

Final contents:

300

400

500

600

700

800

900

1000

1100

1200

10. Expected Output

Latest Transactions:

300

400

500

600

700

800

900

1000

1100

1200

11. Validation

The redesigned system can be tested by inserting more transactions than the capacity.

Test

Capacity:

10 transactions

Transactions inserted:

12 transactions

Expected:

Only the latest 10 transactions remain.

Actual:

300 to 1200

Therefore, the requirement is satisfied.

12. Complexity Comparison

Operation	**Original Array**	**Circular Buffer**
Insert when not full	O(1)	O(1)
Insert when full	O(n)	O(1)
Display	O(n)	O(n)
Space	O(n)	O(n)

The major improvement is insertion when the buffer is full.

13. Why is the Circular Buffer Better?

The original program shifts multiple elements.

The circular buffer does not shift the elements.

Instead, it changes the position of the starting element and overwrites the oldest transaction.

Therefore:

Original full insertion → O(n)

Circular buffer insertion → O(1)

This is more efficient for systems that continuously receive transactions.

14. Defense of the Design

The circular buffer is suitable because the requirement is to retain only the latest 10 transactions.

There is no need to preserve transactions older than the latest 10.

Therefore, overwriting the oldest transaction is acceptable.

The redesigned system:

- Uses fixed memory.
- Avoids unnecessary shifting.
- Maintains the latest transactions.
- Provides constant-time insertion.
- Is simple to implement.
- Is suitable for continuous transaction updates.

15. Alternative Design

Another possible solution is to use:

std::deque<double>

A deque allows efficient insertion and removal from both ends.

Example:

deque<double> transactions;

When the size reaches 10:

transactions.pop_front();

Then:

transactions.push_back(amount);

This is easier to use than manually implementing a circular buffer, although the circular buffer gives direct control over fixed-size storage.

16. Comparison of Designs

Feature	Fixed Array	Circular Buffer	Deque
Fixed capacity	Yes	Yes	Can be controlled
Full insertion	O(n)	O(1)	O(1) at ends
Memory control	High	High	Managed by container
Implementation	Simple	Moderate	Very simple
Suitable for latest records	Yes	Excellent	Excellent

17. Final Result

The limitation of the original Savings Account transaction system was successfully identified.

The original fixed-array design required shifting elements when the array became full. A circular buffer was implemented to solve this problem.

The redesigned system successfully stores the latest 10 transactions while performing insertion in O(1) time.

Therefore, the redesigned solution provides improved efficiency while maintaining the required functionality.

OVERALL CONCLUSION

This assignment successfully demonstrates important advanced C++ and Object-Oriented Programming concepts.

The following concepts were implemented and analysed:

1. Call by value
2. Call by reference
3. Call by pointer
4. Returning references
5. Default arguments
6. Function overloading
7. Classes containing arrays
8. Encapsulation
9. Static data members
10. Static member functions
11. Arrays of objects
12. Friend functions
13. Inline functions
14. Member-function alternatives
15. Time and space complexity
16. Compiler optimization
17. Performance measurement
18. Circular buffer design
19. Program redesign
20. Validation and comparison

The assignment also demonstrates that choosing an appropriate data structure can significantly improve program efficiency. The Savings Account example shows how replacing a shifting fixed-array approach with a circular buffer reduces full-buffer insertion from O(n) to O(1).

FINAL RESULT

The programs and concepts specified in Assignment 2 were successfully implemented, analysed, tested and redesigned where required. The practical exercises demonstrate the use of advanced C++ features and object-oriented programming principles for developing efficient and maintainable programs.